

Sistema de Observabilidad y Disponibilidad de Activos Críticos

Alumno: Brian Andrés Ramírez Ross

Profesor: Ernesto Vivanco

Santiago de Chile
Enero del 2026

índice

1. Problema: La Ceguera Operativa en Edificios Inteligentes	3
2. Solución Propuesta	3
3. Arquitectura y Justificación Metodológica	3
4. Métrica Principal de Éxito	5
5. Impacto y Relevancia	5
6. Próximo Hito: Escalabilidad Cloud Native y DevOps	5
7. Confidencialidad del documento	6

1. Problema: La Ceguera Operativa en Edificios Inteligentes

La gestión de infraestructura moderna sufre de una desconexión crítica entre la capa física y la capa de gestión. Los administradores operan de manera reactiva, detectando fallos de conectividad solo cuando un usuario reporta una incidencia.

La Causa Raíz: Falta de herramientas centralizadas que monitoreen la "salud de la red" en tiempo real.

Consecuencia: Pérdida estimada del 15% en eficiencia operativa y aumento de costos por mantenimiento correctivo de emergencia.

Segmento de Usuario	Jobs to be Done	Pains	Gains
Facility Managers	Garantizar el 99.9% de disponibilidad de servicios.	Detección reactiva de fallos	Monitoreo proactivo y reducción del MTTR.
Asset Managers	Optimizar el valor y gasto operativo del activo.	Costos elevados por mantenimiento correctivo de emergencia.	Extensión de vida útil de equipos mediante alertas tempranas.

2. Solución Propuesta

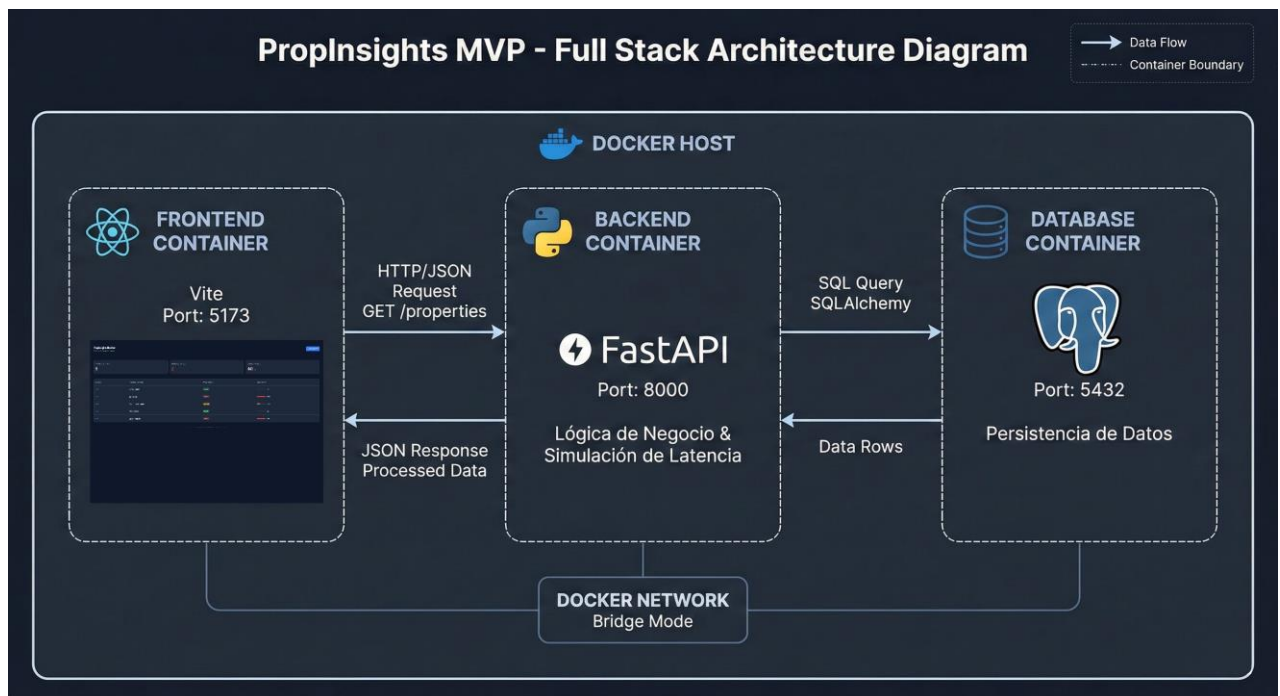
PropInsights es una plataforma Full Stack de monitoreo predictivo. Ingesta señales de latencia de múltiples activos inmobiliarios y, mediante reglas de negocio automatizadas, categoriza el estado operativo en tiempo real, permitiendo una gestión proactiva basada en datos antes de la degradación del servicio.

3. Arquitectura y Justificación Metodológica

Se diseñó una arquitectura desacoplada y contenerizada para garantizar escalabilidad, mantenibilidad y rigor científico en la medición.

- Frontend React + Vite:
 - *Por qué:* Se requiere una interfaz reactiva capaz de renderizar cambios de estado de alta frecuencia (milisegundos) sin recargar el DOM completo.
 - *Cómo:* Implementación de SPA con gestión de estado local para visualización instantánea de alertas.
- Backend FastAPI + Python:
 - *Por qué:* La naturaleza de los datos IoT es asíncrona y de alto volumen. Python ofrece el mejor ecosistema de análisis de datos, y FastAPI permite manejo de concurrencia (async/await) superior a frameworks tradicionales como Flask.

- *Cómo*: API RESTful que procesa la simulación estocástica de latencia y sirve los datos en formato JSON ligero.
- Persistencia (PostgreSQL):
 - *Por qué*: A diferencia de soluciones NoSQL, se priorizó la integridad referencial (ACID) para el inventario de activos críticos, asegurando consistencia en los metadatos.
 - *Cómo*: Modelo relacional orquestado mediante SQLAlchemy ORM.
- Infraestructura (Docker Compose):
 - *Por qué*: Para mitigar la amenaza a la validez interna, asegurando un entorno de ejecución inmutable y reproducible.



4. Métrica Principal de Éxito

- Métrica: Latencia de Red del Percentil 95
- Justificación: Se elige el P 95 en lugar del promedio porque los promedios esconden los picos de latencia outliers que son, precisamente, los indicadores tempranos de fallos en sistemas distribuidos.
- Objetivo del MVP: Visualización y alerta de anomalías >800ms en un tiempo < 1 segundo.

Métrica	Definición	Unidad	Ventana Objetivo	/ Fuente
Latencia P95	Tiempo de respuesta del 95% de los sensores.	ms	< 200 ms Detección Real-time	Backend Logs
Error Rate	Tasa de fallos en la ingesta de telemetría.	%	< 0.1\%	API Middleware
Status Accuracy	Precisión de la clasificación Healthy/Critical.	%	100\% Basado en reglas	Business Logic

5. Impacto y Relevancia

- Operacional: Transición de un modelo de mantenimiento correctivo a uno predictivo, reduciendo el tiempo de inactividad de servicios críticos.
- Económico: Reducción de costos operativos al evitar despliegues técnicos innecesarios mediante triaje remoto.

6. Próximo Hito: Escalabilidad Cloud Native y DevOps

Para soportar la carga masiva de producción (>100,000 sensores), el roadmap técnico evoluciona de la siguiente manera:

- 6.1 Orquestación Avanzada: Migración de Docker Compose a un clúster de Kubernetes (K8s) para alta disponibilidad, implementando CSI Drivers para almacenamiento persistente desacoplado de los pods.
- 6.2 Despliegue Continuo (CI/CD): Implementación de pipelines de GitOps (ArgoCD) para sincronización automática de infraestructura y despliegue sin tiempo de inactividad (Zero Downtime Deployment).
- 6.3 Observabilidad Distribuida: Integración de un stack ELK o Prometheus/Grafana para trazabilidad completa de microservicios.

7. Confidencialidad del documento

Este documento se basa en un proyecto real desarrollado en la empresa del autor y ha sido elaborado exclusivamente con fines académicos para el programa de Magíster en Ingeniería en Informática de la Universidad Andrés Bello. La información contenida en este informe es confidencial y no deberá ser utilizada con fines comerciales, divulgada o reproducida sin la debida autorización.